

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Математические основы интеллектуальных систем»
на тему:

**БАЗА ЗНАНИЙ СИСТЕМЫ АВТОМИЗАЦИИ
ПРОВЕРКИ СТУДЕНЧЕСКИХ РАБОТ**

БГУИР КП4 6-05-0611-03 XXX ПЗ

Студент гр. 421701
Проверяющий

Б. В. Анкуда
Н. А. Гуменный

Минск 2026

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет
Специальность

ИТиУ
6-05-0611-03

Кафедра

ИИТ

УТВЕРЖДАЮ

_____ Зав. кафедрой
«____» _____ 2026 г.

ЗАДАНИЕ

по курсовому проекту студента

Фамилия Имя Отчество в родительном падеже

(фамилия, имя, отчество)

1. Тема работы: База знаний ...

2. Срок сдачи студентом законченной работы: 11.05.2026

3. Исходные данные к проекту:

4. Содержание пояснительной записки (перечень подлежащих разработке вопросов):

Введение

1 Анализ подходов к разработке

2 Проектирование

3 Разработка

Заключение

5. Перечень графического материала (с точным указанием обязательных чертежей):
Компьютерная презентация курсового проекта _____

КАЛЕНДАРНЫЙ ПЛАН

п/п	Наименование этапов курсовой работы	Объем этапа, %	Срок выполнения этапов	Примечание
1	Подбор и изучение литературы	10	9.02 – 20.02	
2	Изучение проблемной области, средств проектирования и разработки	10	21.02 – 27.03	
3	Определение требований к базе знаний системы	10	28.02 – 06.03	
4	Проектирование модели базы знаний системы	25	07.03 – 13.04	
5	Реализация базы знаний системы	30	14.04 – 08.05	
6	Оформление пояснительной записки	10	01.05 – 11.05	
7	Оформление графической части проекта	5	06.05 – 11.05	

Дата выдачи задания: 20.02.2026 г. Проверяющий _____ И. О. Фамилия

Задание принял к исполнению _____ И. О. Фамилия

СОДЕРЖАНИЕ

Перечень условных обозначений	4
Введение	5
1 Анализ предметной области и требований к базе знаний	6
1.1 Характеристика предметной области	6
1.2 Типы задач и сценарии использования	6
1.3 Анализ существующих решений и баз знаний	8
1.4 Характеристика аналогичных решений	9
1.5 Анализ требований к базе знаний	10
1.6 Требования к видам знаний и формализму	11
1.7 Вывод по разделу анализа	12
2 Проектирование модуля предварительного анализа и обработки документов	13
2.1 Общая архитектура интеллектуальной системы и место модуля предварительного анализа	13
2.2 Выбор формализма и логической основы	17
2.3 Концептуальная модель данных модуля предварительного анализа	19
2.4 Логическая и физическая структура модуля предварительного анализа	21
2.5 Моделирование видов данных и событий в модуле предварительного анализа	23
2.6 Обеспечение согласованности, расширяемости и сопровождаемости	25
2.7 Вывод по разделу проектирования	26
3 Реализация базы знаний и средств её использования	27
3.1 Выбор средств реализации	27
3.2 Реализация структуры и наполнения базы знаний	27
3.3 Реализация механизмов доступа и использования базы знаний	27
3.4 Демонстрация работы базы знаний	27
3.5 Методика тестирования	27
3.6 Тестовый набор и классификация тестов	27
3.7 Оформление и результаты тестирования	27
3.8 Анализ качества базы знаний	27
3.9 Вывод по разделу реализации	27
Заключение	28
Список использованных источников	29

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

<i>БЗ</i>	– база знаний
<i>ВК</i>	– вики-система
<i>ЭС</i>	– экспертная система
<i>СУЗ</i>	– система управления знаниями
<i>СП</i>	– сообщество практики
<i>AI</i>	– искусственный интеллект
<i>IT</i>	– информационные технологии
<i>KPI</i>	– ключевые показатели эффективности
<i>KM</i>	– управление знаниями
<i>ROI</i>	– возврат инвестиций

ВВЕДЕНИЕ

Современное образование характеризуется активным внедрением цифровых технологий, направленных на повышение эффективности учебного процесса и сокращение времени на выполнение рутинных задач. Одной из таких задач является проверка студенческих работ, которая требует от преподавателя значительных временных затрат. Необходимо анализировать содержание документов, проверять их структуру, соответствие требованиям оформления и фиксировать найденные нарушения. В связи с этим актуальной является разработка интеллектуальных систем, способных автоматизировать отдельные этапы проверки учебных материалов.

Важной частью такой системы является база знаний, в которой хранятся правила, параметры и ограничения, используемые при анализе студенческих работ. Наличие базы знаний позволяет формализовать требования к оформлению документов, обеспечить единообразие проверки и упростить последующее расширение системы. Разработка базы знаний особенно важна в задачах, связанных с классификацией документов, диагностикой нарушений, выбором правил проверки и объяснением полученных результатов.

Целью данного курсового проекта является разработка базы знаний интеллектуальной системы автоматизации проверки студенческих работ.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области и определить основные задачи, решаемые системой;
- выявить требования к базе знаний;
- определить виды знаний и выбрать способ их представления;
- спроектировать структуру базы знаний;
- реализовать базу знаний и продемонстрировать её использование;
- провести тестирование и оценить результаты работы.

Объектом исследования является интеллектуальная система автоматизации проверки студенческих работ. Предметом исследования является база знаний данной системы, включающая правила проверки, способы представления знаний, структуру хранения и особенности использования при анализе документов.

В рамках курсового проекта рассматривается разработка базы знаний, предназначенной для работы с электронными документами в форматах DOCX и PDF. Система должна поддерживать задачи извлечения текста, применения правил проверки, выявления нарушений и формирования результатов анализа. При этом за рамками проекта остаются сложные механизмы интеллектуального вывода, полнофункциональные сетевые сервисы и расширенные средства интеграции с внешними информационными системами.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ТРЕБОВАНИЙ К БАЗЕ ЗНАНИЙ

1.1 Характеристика предметной области

Предметной областью разрабатываемой системы является автоматизация проверки студенческих работ, представленных в электронном виде. К таким работам относятся лабораторные работы, курсовые работы, отчёты и другие учебные документы. Основная цель системы состоит в упрощении процесса проверки и сокращении времени, затрачиваемого преподавателем на анализ работ.

Основными объектами предметной области являются студенческая работа, файл документа, извлечённый текст, правила проверки и результаты анализа. Документ загружается в систему в формате DOCX или PDF, после чего из него извлекается текстовое содержимое. Далее этот текст анализируется по заданным правилам, связанным с оформлением и структурой работы.

К основным задачам предметной области относятся загрузка документов, извлечение текста, применение правил проверки и отображение результатов анализа. Ожидаемым результатом работы системы является выявление нарушений и представление преподавателю удобной информации о состоянии проверяемой работы.

Особенностями предметной области являются необходимость работы с документами различной структуры, требование к точности извлечения текста, а также использование методических указаний и правил оформления в качестве источников знаний [1]. Кроме того, система должна обеспечивать приемлемое время обработки и наглядное представление результатов проверки [2].

1.2 Типы задач и сценарии использования

База знаний разрабатываемой системы предназначена для решения задач, связанных с автоматизированной проверкой студенческих работ. В рамках проекта она используется для хранения и применения правил, необходимых при анализе загруженных документов.

К основным типам задач, решаемых с использованием базы знаний, относятся классификация, диагностика, выбор и объяснение результатов проверки [3].

Задача классификации заключается в определении типа проверяемого документа и выборе соответствующего набора правил. Например, в зависимости от вида работы система может использовать разные требования к структуре, объёму и оформлению документа.

Задача диагностики связана с выявлением нарушений в оформлении и структуре студенческой работы. На основе правил, хранящихся в базе знаний, система определяет наличие отклонений от установленных требований, таких как отсутствие обязательных разделов, неправильное оформление заголовков, несоответствие параметров текста или другие ошибки.

Задача выбора состоит в определении правил и параметров, которые должны быть применены к конкретному документу. Выбор зависит от типа работы, заданных требований преподавателя или кафедры, а также особенностей конкретного сценария проверки.

Задача объяснения заключается в формировании понятного для пользователя результата проверки. Система должна не только выявлять нарушения, но и указывать, какое именно правило было нарушено, а также представлять описание ошибки в удобной форме.

С использованием базы знаний могут быть реализованы следующие сценарии:

а) Проверка загруженной работы. Пользователь загружает документ в формате DOCX или PDF, после чего система извлекает текст и передаёт данные модулю анализа. Далее выполняется обращение к базе знаний, из которой выбираются правила, соответствующие данному типу работы. В результате пользователь получает информацию о найденных нарушениях и итогах проверки.

б) Использование наборов требований. Пользователь или администратор задаёт параметры проверки, например, выбирает кафедру, дисциплину или тип работы. После этого система обращается к базе знаний и применяет только тот набор правил, который соответствует заданным условиям. Результатом является более точная и адаптированная проверка документа.

в) Формирование итогового отчёта. После завершения анализа система использует сведения из базы знаний для интерпретации выявленных нарушений и подготовки текстовых сообщений. В результате формируется отчёт, содержащий перечень ошибок и замечаний, представленный в понятной для преподавателя форме.

г) Обновление базы знаний. Администратор или разработчик может добавлять новые правила, изменять существующие требования и уточнять параметры проверки. После обновления базы знаний система получает возможность использовать новые данные без изменения основной логики работы приложения.

База знаний в разрабатываемой системе используется для решения задач классификации, диагностики, выбора и объяснения. Она обеспечивает применение формализованных правил при анализе студенческих работ и позволяет формировать результаты проверки в удобной и понятной для пользователя форме.

1.3 Анализ существующих решений и баз знаний

В области автоматизации проверки студенческих работ существует ряд программных решений, предназначенных для анализа текстовых документов, проверки оригинальности, применения критериев оценивания и формирования отчетов. Такие системы позволяют сократить время проверки, повысить единообразие оценки и упростить работу преподавателя.

С точки зрения решаемых задач можно выделить несколько типовых подходов к автоматизации проверки студенческих работ, различающихся назначением системы, способом хранения знаний и формой представления результатов.

Таблица 1.1 – Анализ структур баз знаний существующих решений

Система	Назначение	Преимущества	Ограничения	Применение
Turnitin	Проверка текстов на заимствования и формирование отчетов о совпадениях	Высокая эффективность поиска совпадений, удобные отчеты, широкое применение в образовании	Основной акцент сделан на плагиат, а не на оформление и структуру документа	Высокая для анализа оригинальности
Gradescope	Автоматизация проверки заданий и применение критериев оценивания	Удобство использования рубрик, сокращение времени проверки, поддержка разных типов заданий	Ориентирован в первую очередь на оценивание, а не на проверку оформления документа	Высокая для формализованной оценки
PlagScan	Проверка документов на текстовые совпадения и работа с отчетами	Простота использования, интеграция с образовательными платформами	Ограниченная функциональность в части анализа структуры и форматирования	Средняя для проверки заимствований
Локальные системы проверки документов	Контроль структуры, оформления и параметров текстовых файлов	Возможность адаптации под требования конкретной организации	Ограниченность набора правил и отсутствие универсального подхода	Высокая для прикладных локальных решений

Из приведённого анализа можно сделать вывод, что существующие решения, как правило, ориентированы либо на проверку заимствований, либо на поддержку процесса оценивания. При этом задача комплексной автоматической проверки студенческих работ, включающая анализ структуры документа, параметров оформления и формирование понятного отчёта, реализуется не во всех системах в полном объёме.

1.4 Характеристика аналогичных решений

Существующие системы автоматизированной проверки студенческих работ различаются не только по функциональному назначению, но и по способу организации базы знаний, лежащей в их основе.

Система Turnitin предназначена для выявления текстовых заимствований в документах. Принцип её работы основан на сравнении содержания проверяемого документа с внутренними и внешними источниками. База знаний данной системы имеет преимущественно документный характер и представляет собой крупное хранилище текстовых документов, их представлений, а также метаданных и результатов анализа совпадений. Такая структура эффективна для поиска заимствований, однако менее удобна для хранения формализованных правил оформления студенческих работ [4].

Система Gradescope применяется для автоматизации оценивания студенческих работ. В её основе лежит более структурированная база знаний, включающая критерии оценивания, рубрики, параметры заданий и шаблоны проверки. Такая форма представления знаний обеспечивает удобство сопровождения и изменения критериев, а также единообразие оценивания. Однако основное назначение системы связано с процессом оценивания, а не с детальной проверкой структуры и оформления документа [5].

Система PlagScan, аналогично Turnitin, ориентирована на проверку документов на наличие текстовых совпадений. Её база знаний также строится вокруг текстовых источников, результатов сравнения и отчётов о совпадениях. Данный подход обеспечивает высокую эффективность при анализе оригинальности текста, но характеризуется слабой ориентированностью на проверку структуры и форматирования документов [6].

Кроме того, существуют локальные системы проверки документов, разрабатываемые для нужд конкретной организации или учебной задачи. Для таких решений характерно использование таблиц параметров, наборов правил и шаблонов сообщений о нарушениях. В отличие от рассмотренных систем, их базы знаний ориентированы на хранение формализованных требований к оформлению, ограничений по структуре документа и правил проверки. Это делает их наиболее близкими к задачам разрабатываемой системы [7].

Рассмотрим сильные и слабые стороны существующих подходов. Документно-ориентированные базы знаний обладают высокой полнотой в части хранения текстовых материалов и хорошо подходят для поиска совпадений. Их сильной стороной является масштабируемость и удобство анализа текстов, а слабой — недостаточная формализация правил оформления.

Структурированные базы знаний, основанные на рубриках, правилах и параметрах проверки, удобны для сопровождения, расширения и применения в автоматическом анализе. Их преимуществом является воз-

возможность явного описания критериев, однако они требуют предварительной формализации предметной области [8].

Комбинированные подходы, сочетающие хранение текстовых данных, параметров проверки и формализованных правил, являются наиболее подходящими для систем автоматизации проверки студенческих работ. Они позволяют учитывать как содержимое документа, так и его структурные и оформительские характеристики [3].

Проведённый анализ показывает, что для разрабатываемой системы наиболее целесообразно использовать комбинированный подход к построению базы знаний. В такой базе должны храниться правила оформления студенческих работ, параметры проверки, ограничения на структуру документа, сведения о типовых нарушениях и шаблоны формирования отчётов. Это позволит реализовать автоматическую проверку документов не только с точки зрения текстового содержания, но и с точки зрения соответствия установленным требованиям оформления.

1.5 Анализ требований к базе знаний

База знаний системы должна обеспечивать хранение, структурирование и использование правил, необходимых для автоматизированной проверки студенческих работ на соответствие требованиям оформления и критериям анализа текстовых заимствований. Она должна поддерживать применение формализованных правил к загружаемым документам и обеспечивать формирование результатов проверки.

База знаний должна обеспечивать:

- хранение правил оформления студенческих работ, включая требования к структуре документа, форматированию текста, оформлению заголовков, списков, таблиц, рисунков и списка литературы [1];
- хранение критериев проверки текстовых заимствований и параметров оценки оригинальности работы;
- представление правил в формализованном виде, пригодном для автоматической обработки системой [8];
- использование различных наборов правил в зависимости от типа работы (лабораторная работа, курсовая работа, отчёт по практике и др.);
- поддержку настройки правил проверки под требования конкретной кафедры, дисциплины или преподавателя;
- предоставление описания нарушений и соответствующих рекомендаций по их исправлению;
- формирование данных для построения итогового отчёта о результатах проверки;
- обновление и редактирование набора правил без необходимости изменения программного кода системы;

- хранение версий правил и возможность использования актуального набора требований при выполнении проверки;
- поддержку взаимодействия с модулями анализа документа, извлечения текста и визуализации результатов проверки.

К базе знаний предъявляются требования, обеспечивающие корректность, согласованность и эффективность функционирования системы автоматизированной проверки студенческих работ.

База знаний должна удовлетворять следующим требованиям:

- полнота представления знаний: база знаний должна содержать все основные правила и критерии, необходимые для проверки оформления и анализа содержания студенческих работ;
- непротиворечивость: содержащиеся в базе знаний правила не должны конфликтовать друг с другом в пределах одной предметной области или одного типа документа;
- актуальность: должна быть обеспечена возможность своевременного обновления требований в соответствии с действующими методическими указаниями;
- расширяемость: структура базы знаний должна позволять добавление новых правил, категорий требований и типов документов без изменения общей архитектуры системы;
- сопровождаемость: база знаний должна быть удобна для редактирования, проверки и сопровождения со стороны администратора или эксперта предметной области;
- производительность: обращение к базе знаний и применение правил в ходе проверки должны выполняться за приемлемое время;
- настраиваемость: база знаний должна поддерживать адаптацию под различные учебные подразделения и варианты методических требований;
- надёжность: база знаний должна обеспечивать сохранность правил и устойчивость к ошибкам при обновлении и использовании данных;
- понятность интерпретации: правила и результаты их применения должны быть представлены в форме, удобной для объяснения пользователю.

1.6 Требования к видам знаний и формализму

База знаний должна содержать различные виды знаний, необходимые для выполнения автоматической проверки студенческих работ и интерпретации её результатов.

В системе должны использоваться следующие виды знаний:

- декларативные знания: сведения о требованиях к оформлению документов, допустимых параметрах форматирования, обязательных структурных элементах работы и критериях оценки оригинальности текста;
- структурные знания: описание состава документа, взаимосвязей

между его разделами и классификации типов студенческих работ;

- процедурные знания: правила применения критериев проверки, последовательность анализа документа и логика формирования замечаний;
- знания об ограничениях: допустимые диапазоны параметров оформления, обязательные и запрещённые элементы, условия применимости конкретных правил;
- интерпретационные знания: шаблоны пояснений, используемые при формировании сообщений об обнаруженных нарушениях.

Для представления и обработки знаний целесообразно использовать следующие формализмы:

- структурированные записи о правилах проверки;
- таблицы параметров и ограничений для различных типов документов;
- продукционные правила вида «условие – действие»;
- метаданные, определяющие область применимости конкретного правила;
- шаблоны текстовых сообщений для формирования итогового отчёта о результатах проверки.

Используемые формализмы должны обеспечивать возможность формального описания требований к студенческим работам, их автоматического применения при анализе документов, а также удобство расширения и сопровождения базы знаний в процессе эксплуатации системы.

1.7 Вывод по разделу анализа

По результатам анализа можно сделать следующие выводы:

- предметная область связана с автоматизацией проверки студенческих работ в электронном виде;
- база знаний должна использоваться для хранения правил оформления, параметров проверки и условий формирования результатов анализа;
- основными задачами системы являются классификация документов, выявление нарушений и представление результатов проверки;
- анализ аналогов показал необходимость создания решения, поддерживающего не только проверку заимствований, но и контроль структуры и оформления работ;
- при проектировании базы знаний необходимо учитывать требования полноты, актуальности, непротиворечивости, расширяемости и удобства сопровождения;
- в качестве основы для проектирования будут использоваться структурированные правила и параметры, применяемые при автоматическом анализе загружаемых документов.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ ПРЕДВАРИТЕЛЬНОГО АНАЛИЗА И ОБРАБОТКИ ДОКУМЕНТОВ

2.1 Общая архитектура интеллектуальной системы и место модуля предварительного анализа

Интеллектуальная система автоматизации проверки студенческих работ строится по модульному принципу и включает несколько взаимосвязанных подсистем, обеспечивающих полный цикл обработки документа: от его загрузки до формирования итогового отчёта. Общая архитектура системы ориентирована на разделение функций предварительной обработки, интеллектуального анализа, работы с базой знаний и представления результатов.

В состав основных подсистем системы входят:

- модуль предварительного анализа и обработки документов, отвечающий за приём файла, извлечение из него текстового содержимого, выделение базовых признаков, построение предварительной структурной модели и подготовку данных для дальнейшего анализа;
- подсистема загрузки и сохранения документа, обеспечивающая приём файла и его размещение в хранилище документов;
- подсистема определения типа документа и семестра, использующая данные, подготовленные модулем предварительного анализа;
- подсистема проверки требований, сопоставляющая параметры документа с правилами, хранящимися в базе знаний;
- подсистема формирования отчёта, подготавливающая итоговые результаты проверки в форме, удобной для пользователя;
- база знаний правил проверки, содержащая сведения о типах документов, семестрах, требованиях, нарушениях, правилах проверки и шаблонах интерпретации результатов;
- хранилище документов, предназначенное для сохранения исходных файлов;
- хранилище результатов, предназначенное для сохранения результатов анализа и сформированных отчётов;
- модуль пользовательского интерфейса, обеспечивающий взаимодействие пользователя с системой.

Работа системы начинается с действий пользователя в интерфейсе, после чего загруженный файл передаётся в модуль предварительного анализа и обработки документов. Данный модуль выполняет первичную обработку файла: определяет его формат (PDF или DOCX), извлекает текстовое содержимое, выделяет структурные элементы (заголовки, разделы), определяет

набор базовых признаков документа (например, наличие ключевых слов, объём, количество страниц) и формирует стандартизованное представление документа для последующего анализа. На выходе модуль передаёт структурированные данные подсистемам определения типа, семестра и проверки требований. Таким образом, модуль предварительного анализа является связующим звеном между пользовательским вводом и интеллектуальными компонентами системы.

Архитектурная схема верхнего уровня приведена на рисунке 2.1. На данной схеме показаны основные слои системы, интеллектуальный контур, хранилища и место пользовательского интерфейса в общей структуре приложения..

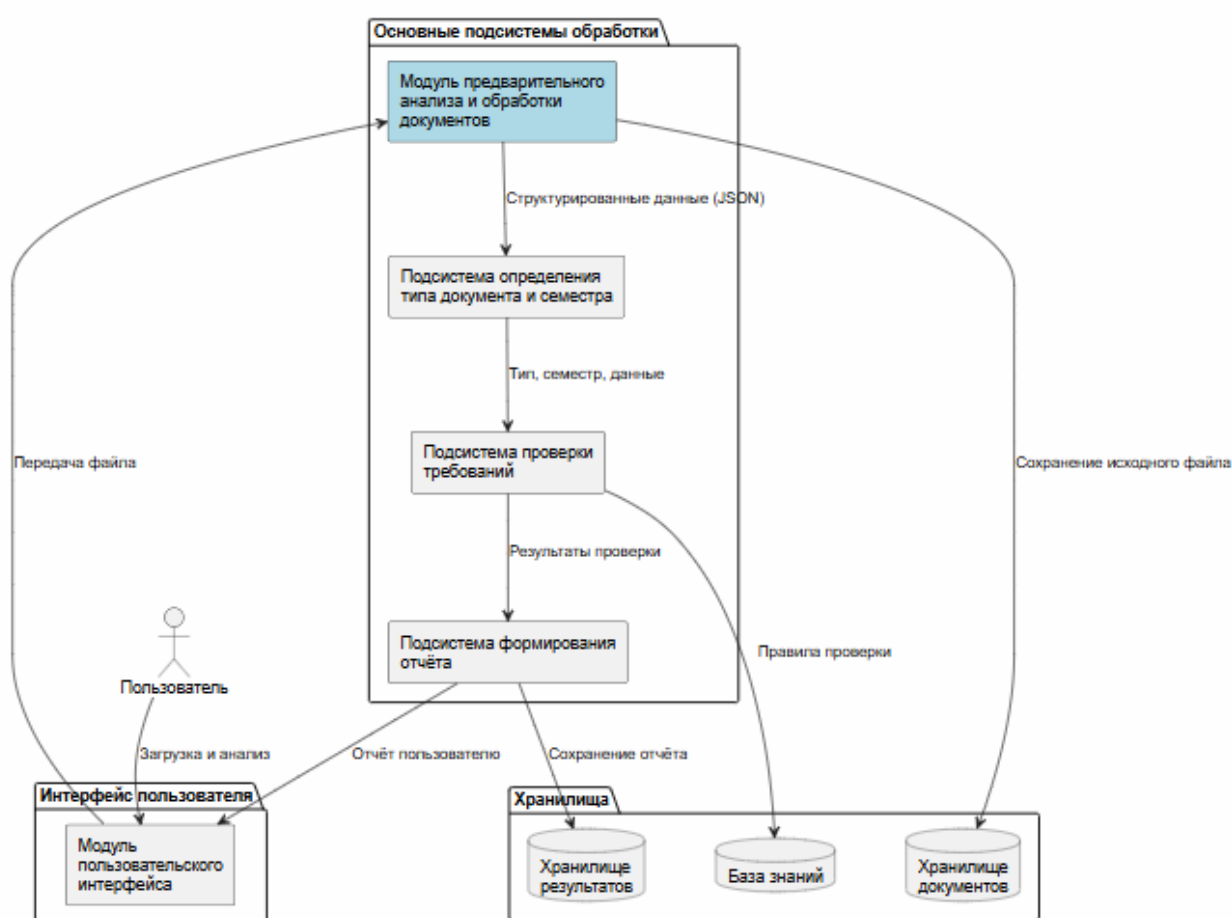


Рисунок 2.1 – Архитектурная схема верхнего уровня

С точки зрения пользователя работа системы определяется набором основных сценариев использования. Студент может загрузить документ, запустить его анализ и получить итоговый отчёт. Преподаватель также может запускать анализ документа и просматривать результаты проверки. При этом сценарий анализа включает вызов внутренних функций определения типа документа, определения семестра и проверки соответствия требовани-

ям. Соответствующая диаграмма вариантов использования приведена на рисунке 2.2.

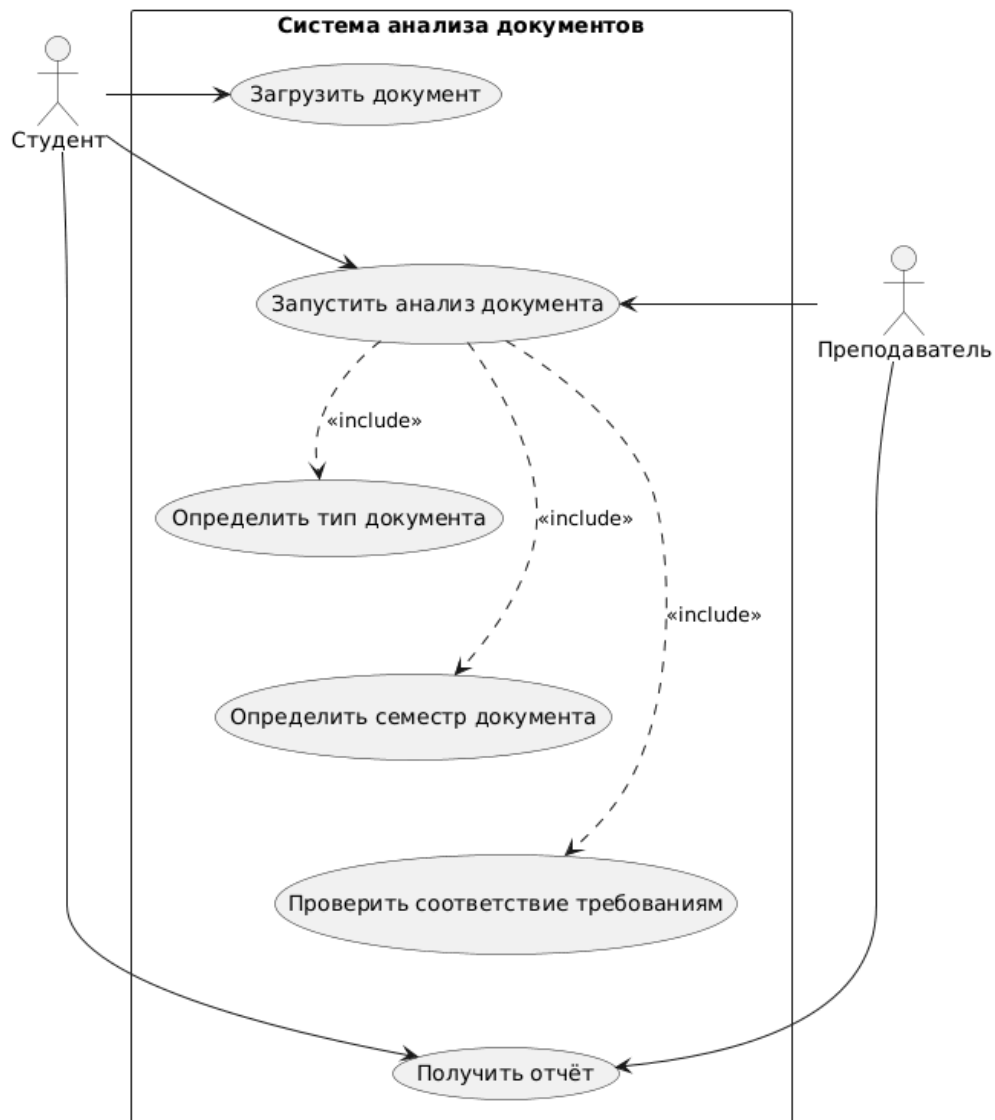


Рисунок 2.2 – Диаграмма вариантов использования системы анализа документов

Временной порядок взаимодействия пользователя с системой, включая этап предварительной обработки документа, показан на диаграмме последовательности, представленной на рисунке 2.3. На диаграмме отражён сценарий выбора и загрузки документа, передачи файла в модуль предварительного анализа, извлечения текста и признаков, сохранения данных и передачи их в подсистемы определения типа и семестра.

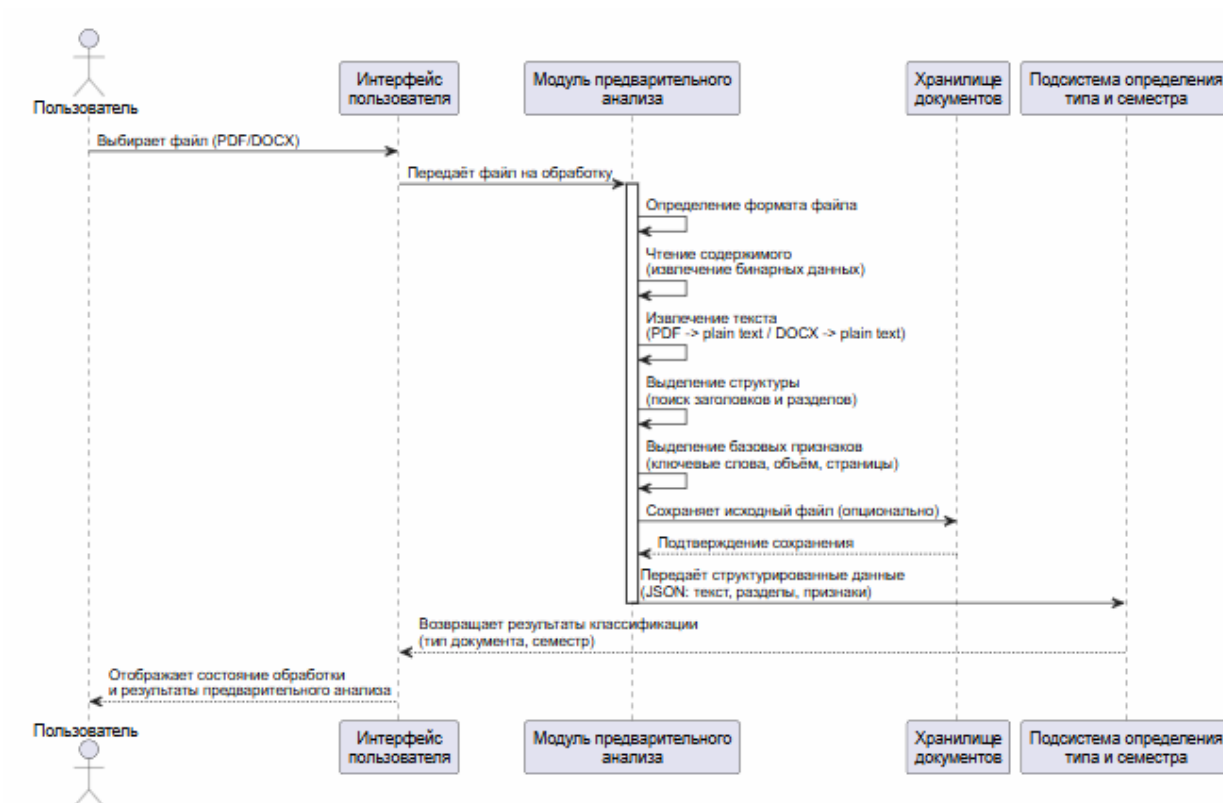


Рисунок 2.3 – Диаграмма последовательности взаимодействия пользователя с системой через модуль предварительного анализа

Более детально внутренняя организация модуля предварительного анализа представлена на диаграмме компонентов, приведённой на рисунке 2.4. Внутри модуля выделяются компонент чтения файла (поддержка форматов PDF и DOCX), компонент извлечения текста, компонент выделения структурных элементов (заголовки, разделы), компонент выделения базовых признаков (ключевые слова, объём, метаданные) и компонент формирования выходной структуры данных. Через указанные компоненты модуль взаимодействует с подсистемами загрузки документа, анализа и формирования отчёта, обеспечивая подготовку данных для последующих этапов.

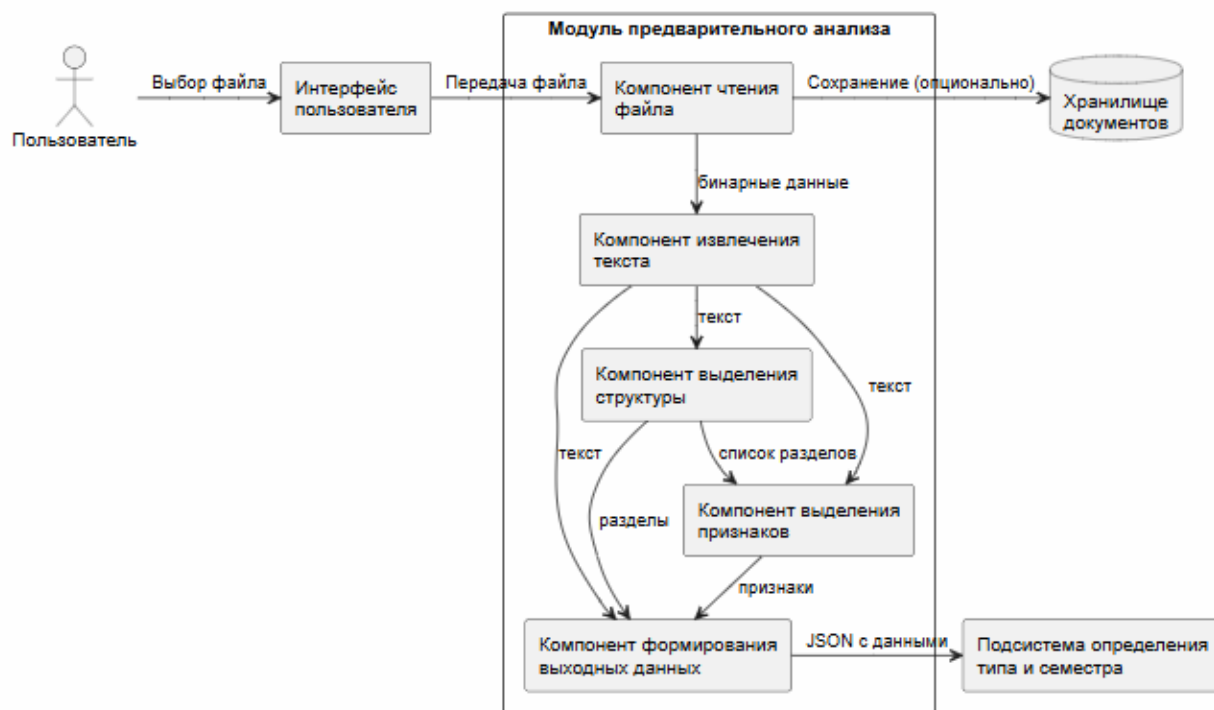


Рисунок 2.4 – Диаграмма компонентов модуля предварительного анализа

Таким образом, общая архитектура интеллектуальной системы автоматизации проверки студенческих работ основана на разделении функций между подсистемами, а модуль предварительного анализа и обработки документов выполняет критически важную задачу подготовки данных, обеспечивая корректное извлечение информации из файлов различных форматов и формирование единого представления документа для последующего интеллектуального анализа.

2.2 Выбор формализма и логической основы

Описание выбранного формализма

При проектировании модуля предварительного анализа используется комбинированный подход к представлению данных и взаимодействию, включающий:

- структурированное объектное представление документа как основную внутреннюю модель обрабатываемых данных;
- формат JSON для обмена данными между модулем предварительного анализа и другими подсистемами системы;
- функциональную модель обработки (конвейер), определяющую последовательность преобразования входного файла в выходную структуру;

– исходные документы в форматах PDF и DOCX как входные объекты, принимаемые модулем.

Выбор такого подхода обусловлен следующими причинами. Модуль предварительного анализа не является хранилищем знаний и не выполняет логического вывода, однако он должен обеспечивать корректное извлечение текстовой и структурной информации из документов различных форматов. Внутреннее представление документа в виде объекта с набором атрибутов (текст, название, формат, разделы, признаки) позволяет унифицировать дальнейшую обработку. Использование JSON в качестве транспортного формата обеспечивает слабую связанность модуля с остальными частями системы и упрощает интеграцию. Функциональная модель обработки описывает конвейер преобразований, что облегчает тестирование и модификацию отдельных этапов (например, замену библиотеки для чтения PDF).

Синтаксис и семантика

В рамках выбранного подхода используются следующие основные синтаксические и семантические элементы:

- *документ* — основной объект обработки, характеризуемый набором атрибутов;
- *атрибуты документа*: имя файла, формат, извлечённый текст, список разделов (с заголовками и границами), набор признаков (например, наличие ключевых слов, количество страниц, объём текста);
- *раздел* — структурная единица документа, включающая заголовок и диапазон текста;
- *признак* — пара «название признака — значение», например `has_table_of_contents: true, semester_keywords: ["4 семестр", "четвёртый семестр"]`;
- *этап обработки* — функциональный блок, принимающий данные определённого формата и возвращающий преобразованные данные;
- *ошибка обработки* — специальный тип события, информирующий о невозможности корректного завершения одного из этапов.

Семантика данного подхода заключается в следующем. Входной файл подаётся на вход конвейера обработки. Каждый этап конвейера выполняет строго определённое преобразование. Например, этап чтения файла принимает путь к файлу и возвращает бинарное содержимое и определённый формат; этап извлечения текста принимает бинарное содержимое и формат, возвращает строку текста; этап выделения структуры принимает текст и возвращает список разделов; этап выделения признаков принимает текст и структуру и возвращает словарь признаков. Результатом работы всего конвейера является объект документа в формате JSON, содержащий все необходимые для дальнейшего анализа поля.

Для обеспечения корректности обработки вводятся следующие правила:

- каждый поддерживаемый формат (PDF, DOCX) должен иметь соответствующий обработчик, гарантирующий извлечение текста;
- структура выходного JSON-объекта должна быть фиксированной и документированной;
- в случае невозможности извлечения текста (например, защищённый паролем PDF) модуль должен генерировать ошибку с понятным описанием;
- выделенные разделы должны содержать не только заголовок, но и указание на позицию в тексте (например, номера строк или символьные смещения) для возможности последующей проверки;
- набор признаков должен быть расширяемым без изменения общей логики конвейера.

Взаимодействие между модулем предварительного анализа и внутренними подсистемами строится на согласовании структуры выходных данных. Например, подсистема определения типа документа ожидает получить объект, содержащий поле **text** и поле **features**, где **features** включает ключевые слова, характерные для разных типов работ. Благодаря такому подходу модуль предварительного анализа остаётся независимым от деталей реализации последующего анализа, но при этом предоставляет всю необходимую информацию.

Таким образом, логическая основа модуля предварительного анализа определяется выбранным формализмом структурированного представления документа, конвейерной моделью обработки и согласованным форматом обмена данными в JSON. Это обеспечивает надёжное извлечение информации из файлов различных форматов и подготовку данных для интеллектуальной части системы.

2.3 Концептуальная модель данных модуля предварительного анализа

Концептуальная модель данных модуля предварительного анализа отражает основные сущности, с которыми работает модуль, и их взаимосвязи. В отличие от общей модели предметной области, данная ER-диаграмма ориентирована на объекты, участвующие в процессе чтения, извлечения и первичной структуризации документа.

В состав модели входят сущности: «Документ», «Формат», «ТекстовоеСодержимое», «Раздел», «Признак». Такое представление позволяет описать данные, формируемые модулем, и их внутреннюю структуру.

Центральной сущностью модели является «Документ», поскольку именно он выступает основным объектом обработки. Документ характеризуется именем файла и имеет определённый формат (PDF или DOCX). Из документа извлекается текстовое содержимое, которое в дальнейшем

используется для выделения структурных элементов и признаков.

С сущностью «Документ» связаны:

- одна сущность «Формат» (документ имеет ровно один формат);
- одна сущность «ТекстовоеСодержимое» (результат извлечения текста);
- множество сущностей «Раздел» (документ может быть разбит на несколько разделов);
- множество сущностей «Признак» (документ может обладать набором признаков).

На рисунке 2.5 представлена ER-диаграмма данных модуля предварительного анализа.

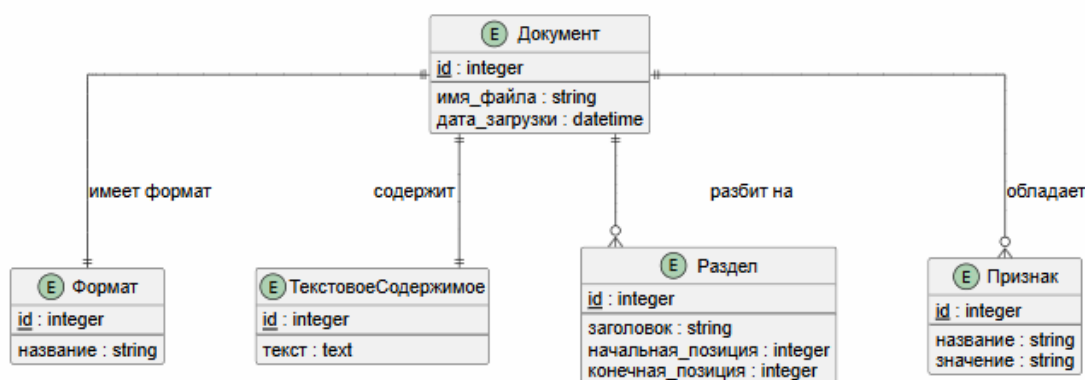


Рисунок 2.5 – ER-диаграмма данных модуля предварительного анализа

Основные бизнес-правила, вытекающие из модели:

- каждый документ должен иметь ровно один формат (PDF или DOCX);
- из документа должно быть извлечено текстовое содержимое (если извлечение невозможно, документ не считается обработанным);
- выделение разделов и признаков является опциональным, но желательным для последующего анализа;
- признаки могут быть как бинарными (наличие/отсутствие), так и числовыми или строковыми.

Представленная модель поддерживает задачи, решаемые модулем предварительного анализа: чтение файла, извлечение текста, построение структуры разделов, выделение признаков и формирование выходной структуры данных. Сущность «Документ» используется для хранения метаданных, «ТекстовоеСодержимое» — для передачи текста, «Раздел» — для описания структурных элементов, «Признак» — для дополнительной информации, полезной при классификации.

Таким образом, концептуальная модель данных модуля предварительного анализа описывает структуру объектов, формируемых в процессе

обработки документа, и служит основой для проектирования логической и физической структуры модуля.

2.4 Логическая и физическая структура модуля предварительного анализа

Логическая структура

Логическая структура модуля предварительного анализа строится как последовательность функциональных блоков (конвейер обработки), каждый из которых отвечает за определённый этап преобразования данных. Такое разделение позволяет изолировать операции чтения, извлечения текста, структурного анализа и выделения признаков друг от друга, что упрощает сопровождение и тестирование модуля.

В логической структуре модуля выделяются следующие основные компоненты:

- *компонент определения формата*, отвечающий за идентификацию типа файла (по расширению и/или сигнатуре);
- *компонент чтения файла*, обеспечивающий загрузку содержимого файла в память в зависимости от формата;
- *компонент извлечения текста*, выполняющий преобразование содержимого PDF или DOCX в plain text;
- *компонент выделения структуры*, осуществляющий поиск заголовков, разделов и формирование иерархии документа;
- *компонент выделения признаков*, вычисляющий набор характеристик документа на основе текста и структуры;
- *компонент формирования выходных данных*, упаковывающий все полученные результаты в JSON-объект установленного формата.

Каждый из указанных компонентов работает с определённым видом данных. Компонент определения формата принимает путь к файлу и возвращает идентификатор формата. Компонент чтения принимает путь и формат, возвращает бинарное содержимое. Компонент извлечения текста принимает бинарное содержимое и формат, возвращает строку текста. Компонент выделения структуры принимает текст, возвращает список разделов. Компонент выделения признаков принимает текст и список разделов, возвращает словарь признаков. Компонент формирования выходных данных собирает все результаты в единый объект.

Связь между компонентами осуществляется через передачу данных в явном виде. Каждый компонент является независимым и может быть заменён или модифицирован без влияния на остальные, при условии сохранения интерфейсов (входных и выходных данных).

Границы ответственности компонентов определены чётко: компонент чтения не занимается выделением структуры, компонент выделения струк-

туры не отвечает за извлечение текста. Такое разделение делает структуру модуля прозрачной и пригодной для поэтапного развития (например, добавления поддержки новых форматов или новых типов признаков).

Физическая структура и форматы хранения

На физическом уровне модуль предварительного анализа реализуется как отдельная часть программного проекта, включающая исходный код обработчиков, библиотеки для работы с PDF и DOCX, а также файлы с описанием структуры выходных данных.

Физическая структура модуля может включать:

- каталог `document_processing/`, содержащий основной код модуля;
- подкаталог `readers/`, содержащий классы для чтения PDF и DOCX (например, `pdf_reader.py`, `docx_reader.py`);
- подкаталог `extractors/`, содержащий классы для извлечения текста (`text_extractor.py`);
- подкаталог `analyzers/`, содержащий классы для выделения структуры (`structure_analyzer.py`) и признаков (`feature_extractor.py`);
- подкаталог `models/`, содержащий определения структур данных (например, `document.py` с классом `Document`);
- подкаталог `utils/`, содержащий вспомогательные функции (например, для работы с регулярными выражениями при поиске заголовков);
- файл `pipeline.py`, реализующий последовательность вызова компонентов.

В работе модуля используются следующие форматы данных:

- PDF и DOCX — как входные документы, принимаемые модулем;
- plain text (строка) — промежуточное представление текста документа;
- JSON — основной формат выходных данных модуля;
- внутренние структуры данных языка программирования (классы/словари) для передачи данных между компонентами внутри модуля.

Физическая организация модуля ориентирована на разделение ответственности между файлами и каталогами. Это позволяет легко находить и изменять код, относящийся к конкретному этапу обработки. Минимизация дублирования данных обеспечивается тем, что каждый компонент работает только с необходимыми ему данными и не хранит избыточную информацию.

Таким образом, логическая и физическая структура модуля предварительного анализа ориентирована на конвейерную обработку документа, ясное разделение обязанностей между компонентами и формирование стандартизованного выходного представления в формате JSON.

2.5 Моделирование видов данных и событий в модуле предварительного анализа

Декларативные данные

В рамках модуля предварительного анализа декларативные данные представлены в виде структурированных сведений о документе, которые формируются в процессе обработки и передаются другим подсистемам. Эти данные описывают статические характеристики документа, полученные в результате анализа его содержимого.

К декларативным данным, формируемым модулем, относятся:

- метаданные документа: имя файла, формат, дата обработки;
- извлечённый текст документа в виде единой строки;
- список разделов с указанием заголовков и границ в тексте;
- набор признаков, таких как количество страниц, наличие оглавления, ключевые слова, предполагаемый семестр (на основе эвристик) и т.д.

Примерами декларативных утверждений, которые могут быть получены модулем, являются:

- документ имеет формат PDF;
- извлечённый текст содержит 15 000 символов;
- в документе обнаружены разделы «Введение», «1 Анализ», «2 Проектирование»;
- документ содержит ключевые слова «пояснительная записка», «курсовой проект»;
- предположительный семестр – 4.

Эти данные не зависят от текущего состояния системы (кроме самого документа) и используются последующими подсистемами для принятия решений (классификация, выбор правил проверки и т.п.).

Процедурные данные и события

Процедурная составляющая модуля предварительного анализа представлена в виде последовательности операций (конвейера) и событий, возникающих в процессе обработки. Основное внимание уделяется правилам преобразования входного файла в выходную структуру данных.

К основным процедурам и событиям, реализуемым в модуле, относятся:

- определение формата файла;
- чтение содержимого файла;
- извлечение текста из содержимого;
- выделение структуры (поиск заголовков по шаблонам);
- вычисление признаков (подсчёт страниц, поиск ключевых слов);
- формирование итогового JSON-объекта;
- обработка ошибок на любом этапе (например, повреждённый файл,

невозможность извлечения текста).

Ключевые правила функционирования модуля можно сформулировать следующим образом:

- если файл имеет расширение **.pdf**, то применяется компонент чтения PDF; если **.docx** — компонент чтения DOCX;
- если формат не поддерживается, генерируется ошибка и обработка прекращается;
- после успешного извлечения текста запускается компонент выделения структуры, который ищет строки, соответствующие шаблонам заголовков (например, начинающиеся с цифры и точки или со слова «Введение»);
- затем компонент выделения признаков анализирует текст и структуру, вычисляя количественные и качественные характеристики;
- наконец, все собранные данные упаковываются в JSON-объект и возвращаются вызывающей стороне.

Процедурная логика работы модуля связана с диаграммой состояний документа, приведённой на рисунке 2.6. Диаграмма отражает последовательность переходов документа между состояниями в процессе обработки модулем предварительного анализа: файл загружен, формат определён, текст извлечён, структура выделена, признаки вычислены, выходные данные сформированы. Для модуля такая модель важна тем, что позволяет формально связать изменения состояния с выполнением конкретных этапов конвейера.

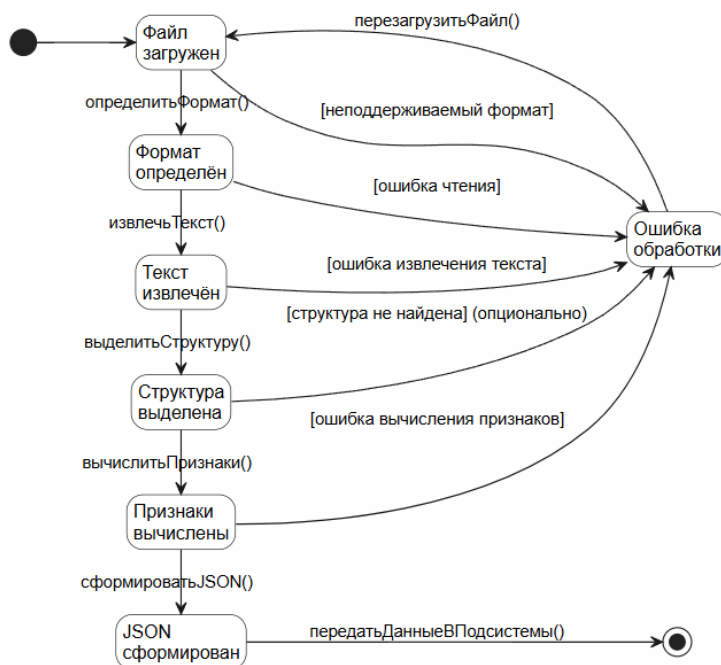


Рисунок 2.6 – Диаграмма состояний документа в модуле предварительного анализа

Однозначность описания процедур обеспечивается тем, что каждый

этап конвейера имеет чётко определённые входные и выходные данные, а также критерии успешного завершения. Например, этап выделения структуры считается успешным, если найден хотя бы один раздел или если документ не содержит явных заголовков (тогда возвращается пустой список разделов). Это позволяет модулю предсказуемо реагировать на различные входные файлы.

Таким образом, в модуле предварительного анализа декларативные данные описывают статические характеристики документа, а процедурные события и правила задают порядок преобразования исходного файла в структурированное представление, пригодное для дальнейшего интеллектуального анализа.

2.6 Обеспечение согласованности, расширяемости и сопровождаемости

Согласованность работы модуля предварительного анализа обеспечивается за счёт строгой типизации данных, передаваемых между компонентами конвейера, и фиксированной структуры выходного JSON-объекта. Для каждого этапа обработки определён контракт: какие данные поступают на вход и какие должны быть получены на выходе.

Контроль согласованности обеспечивается следующими механизмами:

- проверка формата файла перед попыткой чтения;
- обработка исключений, возникающих при работе с библиотеками чтения PDF/DOCX, и преобразование их в унифицированные ошибки модуля;
- валидация выходного JSON на соответствие JSON-схеме перед передачей другим подсистемам;
- использование единых структур данных (классов) внутри модуля для представления документа, раздела, признака;
- логирование каждого этапа обработки для возможности диагностики проблем.

Расширяемость модуля обеспечивается его конвейерной архитектурой. Добавление поддержки нового формата (например, ODT) требует только реализации нового компонента чтения и, возможно, компонента извлечения текста, без изменения остальных частей. Добавление нового признака требует модификации только компонента выделения признаков и, при необходимости, расширения JSON-схемы.

Для расширения модуля достаточно:

- создать новый класс-обработчик, реализующий требуемый интерфейс (например, `BaseReader`);
- зарегистрировать его в фабрике обработчиков;

– при необходимости дополнить модель данных новыми полями (обратная совместимость должна сохраняться).

Сопровождаемость модуля обеспечивается разделением функций между компонентами и каталогами. Каждый компонент имеет чётко очерченную зону ответственности, что упрощает поиск и исправление ошибок. Например, если проблема связана с извлечением текста из PDF, изменения будут локализованы в файле `pdf_reader.py` или `text_extractor.py`.

Эволюция модуля должна выполняться по следующим правилам:

- изменения в одном компоненте не должны нарушать интерфейсы взаимодействия с другими компонентами;
- новые версии выходной JSON-структуры должны быть обратно совместимыми или сопровождаться версионированием;
- все изменения должны покрываться модульными тестами, проверяющими работу конвейера на тестовых документах.

Таким образом, принятая структура модуля предварительного анализа обеспечивает согласованность его работы, удобство сопровождения и возможность последовательного расширения функциональности (добавление новых форматов, новых признаков) без нарушения общей логики обработки.

2.7 Вывод по разделу проектирования

В ходе проектирования модуля предварительного анализа и обработки документов интеллектуальной системы автоматизации проверки студенческих работ было определено его место в общей архитектуре системы, установлены основные связи с подсистемами загрузки, определения типа, семестра и проверки требований, а также выделены ключевые компоненты, обеспечивающие извлечение текста, выделение структуры и признаков документа.

Для проектирования модуля выбран подход, основанный на конвейерной обработке, структурированном представлении документа и обмене данными в формате JSON. Это позволяет унифицировать подготовку данных для последующего анализа независимо от исходного формата файла.

Построенные диаграммы архитектуры, вариантов использования, последовательности, компонентов, ER-модель данных и диаграмма состояний позволили формально описать функциональность модуля и порядок его взаимодействия с остальными частями системы. Принятые проектные решения обеспечивают согласованность, расширяемость и удобство сопровождения модуля предварительного анализа.

3 РЕАЛИЗАЦИЯ БАЗЫ ЗНАНИЙ И СРЕДСТВ ЕЁ ИСПОЛЬЗОВАНИЯ

3.1 Выбор средств реализации

Языки, форматы, библиотеки и инструменты
Обоснование выбора средств

3.2 Реализация структуры и наполнения базы знаний

3.3 Реализация механизмов доступа и использования базы знаний

3.4 Демонстрация работы базы знаний

3.5 Методика тестирования

3.6 Тестовый набор и классификация тестов

3.7 Оформление и результаты тестирования

3.8 Анализ качества базы знаний

3.9 Вывод по разделу реализации

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов / В. Г. Горовой, А. А. Горюшкин, А. В. Грицай, В. А. Пархименко. — Минск : БГУИР, 2021. — 107 с.

[2] Доманов, А. Т. Стандарт предприятия. Дипломные проекты (работы). Общие требования : СТП 01-2017 / А. Т. Доманов, Н. И. Сорока. — Минск : БГУИР, 2017. — 169 с.

[3] Люгер, Д. Ф. Искусственный интеллект: стратегии и методы решения сложных проблем / Д. Ф. Люгер. — М. : Вильямс, 2005. — 864 с. — пер. с англ.

[4] Turnitin [Electronic resource]. — Mode of access: <https://www.turnitin.com>. — Date of access: 10.03.2026.

[5] Gradescope [Electronic resource]. — Mode of access: <https://www.gradescope.com>. — Date of access: 10.03.2026.

[6] PlagScan [Electronic resource]. — Mode of access: <https://www.plagscan.com>. — Date of access: 10.03.2026.

[7] Шункевич, Д. В. Агентно-ориентированные решатели задач интеллектуальных систем: автореф. дисс. ... канд. техн. наук : 05.13.17 / БГУИР. — Минск , 2018. — 27 с.

[8] Джарратано, Д. Экспертные системы: принципы разработки и программирование / Д. Джарратано, Г. Райли. — М. : Вильямс, 2007. — 1152 с. — пер. с англ.